Alterações no código de KNN e SVD

main.py

1. Validação das notas e mínimo de 5 avaliações

Problema: A verificação nunca disparava, porque o usuário tinha acabado de ser inserido. Não havia mínimo de notas nem checagem de IDs, repetidos e escala.

ANTES

```
if USER_ID not in ratings["userId"].unique():
    raise SystemExit(f"O usuário {USER_ID} não existe no dataset.")
```

DEPOIS

```
# Recusa avaliações que os algoritmos não conseguiriam usar, explicando o motivo
def validar_avaliacoes(novas_avaliacoes, movies):

    if len(novas_avaliacoes) < MIN_AVALIACOES:
        raise ValueError(
            f"É preciso avaliar pelo menos {MIN_AVALIACOES} filmes "
            f"({len(novas_avaliacoes)} recebidas)."
        )

    ids = [mid for mid, _ in novas_avaliacoes]  # movieIds informados pelo usuário

    # Notas repetidas para o mesmo filme seriam somadas na matriz do KNN
    repetidos = sorted(mid for mid, qtd in Counter(ids).items() if qtd > 1)
    if repetidos:
        raise ValueError(f"Filmes avaliados mais de uma vez: {repetidos}")

    inexistentes = sorted(set(ids) - set(movies["movieId"]))  # IDs que não estão no catálogo
    if inexistentes:
        raise ValueError(f"movieIds que não existem em {ARQUIVO_FILMES}: {inexistentes}")
```

2. Divisão treino/teste comum aos algoritmos

Problema: A divisão ficava dentro do SVD, e o KNN não era avaliado. Agora é feita uma vez, sobre a base sem o usuário fictício, e vale para os dois.

ANTES

```
trainset, testset = train_test_split(data, test_size=0.2, random_state=42)
```

DEPOIS

```
# Separa a base uma única vez para que todos os algoritmos sejam avaliados nas mesmas avaliações
def dividir_treino_teste(ratings):

    # Constrói o trainset e o testset (20% dos dados para teste)
    return train_test_split(ratings, test_size=PROPORCAO_TESTE, random_state=SEMENTE)
...
# A avaliação usa a base original, sem o usuário fictício, para que as métricas não mudem a cada
    perfil testado
treino_df, teste_df = dividir_treino_teste(ratings_base)  # 80% para treino e 20% para teste

metricas = {}  # RMSE, MAE, fração prevista e tempo de cada algoritmo
for nome, avaliar in [("KNN", avaliar_knn), ("SVD", avaliar_svd)]:
    start = time.time()
    metricas[nome] = avaliar(treino_df, teste_df)
    metricas[nome]["tempo_s"] = time.time() - start

print(f"===== Métricas no conjunto de teste ({len(teste_df)} avaliações) =====")
tabela_metricas = pd.DataFrame(metricas).T.rename_axis("algoritmo").reset_index()  # uma linha por
    algoritmo
print(formatar_tabela(tabela_metricas))
```

3. Cache com invalidação

Problema: Se um CSV fosse trocado, o cache antigo continuava sendo carregado. Um cache gerado em outro computador também podia travar a execução.

ANTES

```
if os.path.exists(CACHE_DADOS):
    print("Carregando ratings e movies do cache local...")
    with open(CACHE_DADOS, "rb") as f:
        dados = pickle.load(f)
        movies = dados["movies"]
        ratings = dados["ratings"]
```

DEPOIS

```
assinatura = assinatura_arquivos()  # versão atual dos CSVs

if os.path.exists(CACHE_DADOS):
    # Um cache gerado em outro computador ou com outra versão do pandas pode não abrir, e nesse caso é
        refeito
    try:
        with open(CACHE_DADOS, "rb") as f:
            dados = pickle.load(f)  # conteúdo salvo no cache
    except Exception:
        dados = {}  # cache ilegível, tratado como desatualizado

    # O cache só é reaproveitado se tiver sido gerado a partir dos mesmos CSVs
    if dados.get("assinatura") == assinatura:
        print("\nCarregando ratings e movies do cache local...")
        return dados["movies"], dados["ratings"]

    print("\nCache desatualizado ou ilegível, recarregando os CSVs...")
else:
    print()  # separa a leitura dos CSVs da linha de início
```

4. Remoção do CACHE_MODELO

Problema: Constante declarada e nunca usada.

ANTES

```
CACHE_MODELO = "svd_model_cache.pkl"
CACHE_DADOS = "dados_processados.pkl"
CORE_GENERO_MIN_FRACAO = 0.6
```

DEPOIS

```
ARQUIVO_FILMES = "movies_small.csv"  # catálogo de filmes usado na execução
ARQUIVO_AVALIACOES = "ratings_small.csv"  # avaliações usadas na execução
CACHE_DADOS = "dados_processados.pkl"  # arquivo de cache dos CSVs
```

5. Fim do global em inserir_novo_usuario

Problema: O global vinha antes da docstring e a anulava. A função passou a receber e devolver o DataFrame.

ANTES

```
def inserir_novo_usuario(novas_avaliacoes):
    global ratings
...
ratings = pd.concat([ratings, novos_registros], ignore_index=True)

return novo_usuario_id
```

DEPOIS

```
def inserir_novo_usuario(ratings, movies, novas_avaliacoes):
...
return pd.concat([ratings, novos_registros], ignore_index=True), novo_usuario_id
```

6. Perfil do usuário em função própria

Problema: O cálculo do perfil ficava solto no bloco principal. Virou montar_calculos, com o mesmo código.

ANTES

```
# Pegamos os filmes já avaliados pelo usuário-alvo
user_ratings = ratings[ratings["userId"] == USER_ID].merge(movies, on="movieId")
watched_ids = set(user_ratings["movieId"])
```

DEPOIS

```
# Pré-calcula o histórico e o perfil de gêneros do usuário, compartilhados por todos os algoritmos
def montar_calculos(movies, ratings, user_id):

    # Pega os filmes já avaliados pelo usuário-alvo
    user_ratings = ratings[ratings["userId"] == user_id].merge(movies, on="movieId")  # avaliações do
        usuário com título e gêneros
    watched_ids = set(user_ratings["movieId"])  # filmes que o usuário já avaliou
```

7. Perfil de teste como lista de avaliações

Problema: A lista ficou separada da chamada da função, e avaliações inválidas passam a encerrar com mensagem clara. O perfil de Animação, que usava um filme inexistente na base (ID 288513), foi retirado.

ANTES

```
"""
Horror
"""
USER_ID = inserir_novo_usuario([
    (2459, 5.0),
    (148671, 5.0),
    (168250, 4.5),
    (168252, 5.0),
    (162082, 4.0),
])
```

DEPOIS

```
# Horror
avaliacoes_usuario = [
    # Pares (movieId, nota) de um usuário que gosta de terror
    (2459, 5.0),
    (148671, 5.0),
    (168250, 4.5),
    (168252, 5.0),
    (162082, 4.0),
]
...
# Avaliações inválidas encerram a execução com uma mensagem clara em vez de um traceback
try:
    ratings, USER_ID = inserir_novo_usuario(ratings_base, movies, avaliacoes_usuario)  # base com o
        usuário fictício e o ID dele
except ValueError as erro:
    raise SystemExit(f"Avaliações inválidas. {erro}")
```

8. Exibição separada do cálculo e tabelas alinhadas à esquerda

Problema: As tabelas saíam alinhadas à direita, e o perfil do usuário era impresso dentro da função de cálculo. Agora as funções só devolvem dados, e a exibição fica no bloco principal.

ANTES

```
print(f"\n===== Filmes avaliados pelo usuário {USER_ID} =====")
print(
    user_ratings[["title", "genres", "rating"]]
    .sort_values("rating", ascending=False)
    .to_string(index=False)
)
```

DEPOIS

```
# Monta uma tabela em texto com todas as colunas alinhadas à esquerda, usada só na exibição do terminal
def formatar_tabela(df):
    textos = df.apply(formatar_coluna)  # cópia da tabela com cada valor já em texto
    larguras = {c: max([len(str(c))] + textos[c].str.len().tolist()) for c in textos.columns}  #
        largura de cada coluna
    linhas = ["  ".join(str(c).ljust(larguras[c]) for c in textos.columns)]  # começa pelo cabeçalho
    linhas += [
        "  ".join(valor.ljust(larguras[c]) for c, valor in zip(textos.columns, linha))
        for linha in textos.itertuples(index=False)
    ]
    return "\n".join(linha.rstrip() for linha in linhas)
...
print(f"===== Filmes avaliados pelo usuário {USER_ID} =====")
print(formatar_tabela(calculos["historico_usuario"]))
```

9. Separadores entre as etapas

Problema: As etapas ficavam coladas no terminal. Agora uma linha separa o carregamento, o KNN, o SVD e as métricas.

ANTES

```
print("\nComeço KNN...")
```

DEPOIS

```
SEPARADOR = "\n"+ ("-" * 100) + "\n"  # linha que separa as etapas na saída do terminal
...
print(f"\nFim do KNN... ({end - start:.4f} segundos)")
print(SEPARADOR)
print("Começo Matriz Fatorizada...")
```

10. Escolha do modo do filtro de gênero

Problema: Não havia como rodar sem o filtro para medir o efeito dele.

ANTES

Não existia.

DEPOIS

```
MODO_FILTRO_GENERO = "core" # "core", "intersecao" ou None, detalhados em filtros.py
```

knn.py

1. Avaliação do KNN

Problema: O KNN não tinha nenhuma métrica e não podia ser comparado ao SVD.

ANTES

Não existia.

DEPOIS

```
# Mede RMSE e MAE do KNN no teste, usando a média do usuário quando nenhum vizinho avaliou o filme
def avaliar_knn(treino, teste):

    matriz, user_index, movie_index = montar_matriz(treino)  # matriz montada só com o treino
    knn = treinar_knn(matriz)  # modelo ajustado só com o treino

    # Médias usadas como previsão de reserva quando os vizinhos não conseguem prever
    media_global = treino["rating"].mean()  # média de todas as notas do treino
    media_usuario = treino.groupby("userId")["rating"].mean()  # média de cada usuário no treino

    # Dicionários evitam buscas lineares nos índices a cada usuário do teste
    pos_usuario = {uid: pos for pos, uid in enumerate(user_index)}  # ID real do usuário para a linha
        da matriz
    pos_filme = {mid: pos for pos, mid in enumerate(movie_index)}  # ID real do filme para a coluna da
        matriz

    erros = []  # diferenças entre previsão e nota real, por usuário
    previstos_por_vizinhos = 0  # quantos pares do teste foram previstos pelos vizinhos
...
erros = np.concatenate(erros)  # todos os erros do teste em um único array

return {
    "rmse": float(np.sqrt(np.mean(erros ** 2))),
    "mae": float(np.mean(np.abs(erros))),
    "fracao_prevista_pelo_modelo": previstos_por_vizinhos / len(erros),
}
```

2. Matriz, modelo e vizinhos em funções

Problema: A avaliação precisava repetir esses passos. Virar função evitou copiar o código.

ANTES

```
# Pegamos os K_VIZINHOS vizinhos mais próximos do usuário-alvo
distancias, indices = knn.kneighbors(
    matriz_avaliacoes[user_pos], n_neighbors=K_VIZINHOS + 1
)

# Removemos o próprio usuário-alvo da lista de vizinhos
vizinhos = [
    (idx, dist)
    for idx, dist in zip(indices[0], distancias[0])
    if idx != user_pos
]
```

DEPOIS

```
# Busca os vizinhos mais parecidos com um usuário, sem ele mesmo e sem os de similaridade nula
def buscar_vizinhos(knn, matriz, user_pos):

    # Pega os K_VIZINHOS vizinhos mais próximos do usuário-alvo, com distâncias e posições do mais
        próximo ao mais distante
    distancias, indices = knn.kneighbors(
        matriz[user_pos], n_neighbors=K_VIZINHOS + 1
    )

    # Remove o próprio usuário-alvo da lista de vizinhos
    mascara = indices[0] != user_pos  # True para todos, menos para o próprio usuário

    # Menores distâncias significam maior similaridade
    similaridades = 1 - distancias[0][mascara]  # similaridade de cada vizinho, de 0 a 1
    vizinhos = indices[0][mascara]  # posição de cada vizinho na matriz

    # Vizinhos sem nenhuma similaridade não dizem nada sobre o gosto do usuário
    positivos = similaridades > 0  # True para os vizinhos que serão mantidos

    return vizinhos[positivos], similaridades[positivos]
...
vizinhos, similaridades = buscar_vizinhos(knn, matriz_avaliacoes, user_pos)  # vizinhos úteis e o peso
    de cada um
```

3. Comentário sobre a similaridade do cosseno

Problema: O texto original dizia que a escala de cada usuário é ignorada, o que só vale com centralização pela média. O comentário foi corrigido.

ANTES

```
# Configuramos o modelo de KNN
# Vamos usar a "similaridade de cosseno" para calcular a distância entre os vetores.
# Isso mede o ângulo entre os perfis de notas, focando nos gostos e ignorando se um usuário é mais
    "chato" ou "generoso" com as notas.
knn = NearestNeighbors(metric="cosine", algorithm="brute")
knn.fit(matriz_avaliacoes)
```

DEPOIS

```
# Configura o modelo de vizinhos mais próximos. Sem centralizar as notas pela média, a escala de cada
    usuário ainda pesa um pouco
knn = NearestNeighbors(
    # Usa a "similaridade de cosseno" para calcular a distância entre os vetores
    metric="cosine",  # Isso mede o ângulo entre os perfis de notas, focando nos gostos em comum
    algorithm="brute",  # compara com todos os usuários, sem índice aproximado
)

knn.fit(matriz)
```

4. Filtro de gênero chamado de filtros.py

Problema: O mesmo filtro estava copiado no KNN e no SVD.

ANTES

```
# Filtramos candidatos pelo perfil de gênero do usuário
# Se o candidato tiver o perfil core do usuário, ele é mantido; caso contrário, é descartado.
if perfil_core_usuario:
    pontuacao_final = {
        movie_id: pontuacao
        for movie_id, pontuacao in pontuacao_final.items()
        if perfil_core_usuario.issubset(genero_por_filme.get(int(movie_id), set()))
    }
```

DEPOIS

```
# Filtra os candidatos pelo perfil de gênero do usuário
pontuacao_final = aplicar_filtro_genero(pontuacao_final, calculos)  # Se o candidato tiver o perfil
    core do usuário, ele é mantido
```

5. Aviso de lista incompleta

Problema: Com menos de 5 candidatos após o filtro, a lista saía menor sem aviso.

ANTES

Não existia.

DEPOIS

```
# Um perfil core muito restrito pode deixar menos candidatos do que o necessário
if len(top_ids) < N_RECOMENDACOES:
    print(f"Aviso: o KNN encontrou apenas {len(top_ids)} candidatos após os filtros.")
```

6. Índice do resultado reiniciado

Problema: Só o SVD reiniciava o índice, e os dois devolviam o resultado em formatos diferentes.

ANTES

```
recomendados = recomendados.sort_values("nota_estimada", ascending=False)
```

DEPOIS

```
recomendados = recomendados.sort_values("nota_estimada", ascending=False).reset_index(drop=True)  #
    lista da maior para a menor nota estimada
```

7. Parâmetros removidos da docstring

Problema: O bloco de parâmetros era igual no KNN e no SVD. Foi retirado, e a descrição da função virou o comentário antes do def.

ANTES

```
def rodar_knn(movies, ratings, USER_ID, calculos):
    """
    Roda o algoritmo KNN para recomendar filmes para o usuário USER_ID.
    movies: DataFrame de filmes
    ratings: DataFrame de avaliações
    USER_ID: id do usuário-alvo
    calculos: dicionário com informações pré-calculadas
    Returning: DataFrame de recomendações
    """
```

DEPOIS

```
# Roda o algoritmo KNN para recomendar filmes para o usuário USER_ID
def rodar_knn(movies, ratings, USER_ID, calculos):
```

matriz_fatorizada.py

1. SVD treinado com todas as notas do usuário

Problema: A divisão 80/20 era aplicada depois de inserir o usuário, então só 3 das 5 notas dele entravam no treino.

ANTES

```
# Construimos o trainset e o testset (20% dos dados para teste)
trainset, testset = train_test_split(data, test_size=0.2, random_state=42)

# Iniciamos e treinamos o modelo SVD
model = SVD(n_factors=20, lr_all=0.01, reg_all=0.01, n_epochs=20, random_state=42)
model.fit(trainset)
```

DEPOIS

```
# Treina o SVD com todas as avaliações recebidas, sem separar teste
def treinar_svd(ratings):

    # Configura o leitor e carrega os dados para o formato do Surprise
    reader = Reader(rating_scale=ESCALA_NOTAS)  # informa ao Surprise a escala das notas
    data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)  # avaliações no
        formato do Surprise

    # Inicia e treina o modelo SVD
    model = SVD(**PARAMETROS_SVD)  # modelo de fatoração de matrizes
    model.fit(data.build_full_trainset())

    return model
```

2. Escala de notas

Problema: O MovieLens vai de 0,5 a 5,0. A escala de 0 a 5 distorcia o limite das previsões e o RMSE.

ANTES

```
reader = Reader(rating_scale=(0,5))
```

DEPOIS

```
ESCALA_NOTAS = (0.5, 5.0) # escala real das notas do MovieLens
...
reader = Reader(rating_scale=ESCALA_NOTAS)  # informa ao Surprise a escala das notas
```

3. Métricas devolvidas pela função

Problema: RMSE e MAE eram calculados, mas só impressos pelo Surprise, sem chegar ao main.

ANTES

```
# Avaliamos o modelo com metricas RMSE e MAE no conjunto de teste
predictions = model.test(testset)
rmse = accuracy.rmse(predictions)
mae = accuracy.mae(predictions)
```

DEPOIS

```
# Avalia o modelo com métricas RMSE e MAE no conjunto de teste
predictions = model.test(testset)  # previsão do modelo para cada avaliação de teste

# Fora do treino, o SVD prevê só com média global e vieses, sem usar os fatores latentes
conhecidos = sum(1 for uid, iid, _ in testset if _esta_no_treino(model.trainset, uid, iid))  # pares
    do teste com usuário e filme presentes no treino

return {
    "rmse": accuracy.rmse(predictions, verbose=False),
    "mae": accuracy.mae(predictions, verbose=False),
    "fracao_prevista_pelo_modelo": conhecidos / len(testset),
}
```

4. Loop de previsão sem iterrows

Problema: O iterrows percorria cerca de 9.700 filmes linha a linha.

ANTES

```
# Filtramos os filmes que o usuário já viu
 candidatos_df = movies[~movies["movieId"].isin(watched_ids)]

 # Geramos as notas estimadas
 pontuacao_final = {
     row["movieId"]: model.predict(USER_ID, row["movieId"]).est
     for _, row in candidatos_df.iterrows()
 }
```

DEPOIS

```
# Filtra os filmes que o usuário já viu
candidatos = movies.loc[~movies["movieId"].isin(watched_ids), "movieId"]  # IDs dos filmes ainda não
    vistos

# Gera as notas estimadas
pontuacao_final = {  # nota estimada de cada filme candidato
    movie_id: model.predict(USER_ID, movie_id).est
    for movie_id in candidatos
}
```

5. Filtro de gênero chamado de filtros.py

Problema: Mesmo filtro copiado do KNN.

ANTES

```
# Filtramos candidatos pelo perfil de gênero do usuário
# Se o candidato tiver o perfil core do usuário, ele é mantido; caso contrário, é descartado.
if perfil_core_usuario:
    pontuacao_final = {
        movie_id: pontuacao
        for movie_id, pontuacao in pontuacao_final.items()
        if perfil_core_usuario.issubset(genero_por_filme.get(int(movie_id), set()))
    }
```

DEPOIS

```
# Filtra os candidatos pelo perfil de gênero do usuário
pontuacao_final = aplicar_filtro_genero(pontuacao_final, calculos)  # Se o candidato tiver o perfil
    core do usuário, ele é mantido
```

6. Parâmetros removidos da docstring

Problema: Mesma mudança do knn.py.

ANTES

```
def rodar_svd(movies, ratings, USER_ID, calculos):
    """
    Roda a Fatoração de Matrizes (SVD) para recomendar filmes para o usuário USER_ID.
    movies: DataFrame de filmes
    ratings: DataFrame de avaliações
    USER_ID: id do usuário-alvo
    calculos: dicionário com informações pré-calculadas
    Returning: DataFrame de recomendações
    """
```

DEPOIS

```
# Roda a Fatoração de Matrizes (SVD) para recomendar filmes para o usuário USER_ID
def rodar_svd(movies, ratings, USER_ID, calculos):
```

filtros.py

1. Arquivo novo com o filtro de gênero

Problema: Uma única função atende o KNN, o SVD e a futura rede neural, com três modos. O modo core repete o filtro original.

ANTES

Não existia.

DEPOIS

```
# Mantém só os candidatos compatíveis com o perfil core do usuário, conforme o modo definido no main
def aplicar_filtro_genero(pontuacao_final, calculos):

    modo = calculos.get("modo_filtro_genero")  # "core", "intersecao" ou None
    perfil_core = calculos["perfil_core_usuario"]  # gêneros que definem o gosto do usuário
    genero_por_filme = calculos["genero_por_filme"]  # conjunto de gêneros de cada filme

    # Sem modo ou sem perfil core definido, todos os candidatos seguem adiante
    if modo is None or not perfil_core:
        return pontuacao_final

    # "core" exige todos os gêneros do perfil e "intersecao" exige só parte deles
    if modo == "core":
        minimo_em_comum = len(perfil_core)  # todos os gêneros do perfil core
    elif modo == "intersecao":
        minimo_em_comum = math.ceil(len(perfil_core) * FRACAO_MIN_INTERSECAO)  # pelo menos metade do
            perfil core (arredondado para cima)
    else:
        raise ValueError(f"Modo de filtro de gênero desconhecido: {modo}")
```

Todos os arquivos

1. Comentário antes de cada função

Problema: Nenhuma função explicava o objetivo antes do def.

ANTES

```
def carregamento_dados():
```

DEPOIS

```
# Carrega filmes e avaliações, reaproveitando o cache enquanto os CSVs não mudarem
def carregamento_dados():
```

2. Comentário em cada variável

Problema: As variáveis não tinham explicação.

ANTES

```
watched_ids = calculos['watched_ids']
perfil_generos_usuario = calculos['perfil_generos_usuario']
perfil_core_usuario = calculos['perfil_core_usuario']
genero_por_filme = calculos['genero_por_filme']
```

DEPOIS

```
watched_ids = calculos['watched_ids']  # filmes que o usuário já avaliou
perfil_generos_usuario = calculos['perfil_generos_usuario']  # todos os gêneros do histórico do usuário
genero_por_filme = calculos['genero_por_filme']  # conjunto de gêneros de cada filme
```

3. Comentários seguidos separados

Problema: Havia comentários colados uns nos outros. Cada um foi para a linha que descreve, e frases quebradas viraram uma linha.

ANTES

```
# Converte a coluna de IDs de usuários para o tipo "category" do Pandas.
# O Pandas mapeia automaticamente cada ID real para um número inteiro sequencial (começando do 0).
user_ids = ratings["userId"].astype("category")
movie_ids = ratings["movieId"].astype("category")
...
# Média ponderada: soma(similaridade * nota) / soma(similaridade)
# Só consideramos filmes com suporte mínimo (evita recomendar algo que só
# 1 vizinho avaliou, o que costuma gerar recomendações estranhas/obscuras)
pontuacao_final = {
    movie_id: scores[movie_id] / soma_similaridades[movie_id]
    for movie_id in scores
    if contagem_vizinhos[movie_id] >= SUPORTE_MINIMO
}
```

DEPOIS

```
# Converte a coluna de IDs de usuários para o tipo "category" do Pandas
user_ids = ratings["userId"].astype("category")  # O Pandas mapeia cada ID real para um número inteiro
    sequencial (começando do 0)
movie_ids = ratings["movieId"].astype("category")  # mesmo mapeamento, aplicado aos IDs de filmes
...
# Nota estimada de cada filme candidato
pontuacao_final = {
    # Média ponderada: soma(similaridade * nota) / soma(similaridade)
    movie_id: scores[movie_id] / soma_similaridades[movie_id]
    for movie_id in scores
    # Só considera filmes com suporte mínimo (evita recomendar algo que só 1 vizinho avaliou)
    if contagem_vizinhos[movie_id] >= SUPORTE_MINIMO
}
```

4. Comentários padronizados na terceira pessoa

Problema: Os comentários misturavam primeira pessoa do plural e terceira pessoa, com e sem ponto final. Todos seguem agora o mesmo padrão.

ANTES

```
# Pegamos a posição do usuário-alvo dentro da matriz
user_pos = np.where(user_index == USER_ID)[0][0]
...
# Selecionamos os top N_RECOMENDACOES filmes recomendados
top_ids = sorted(pontuacao_final, key=pontuacao_final.get, reverse=True)[:N_RECOMENDACOES]
```

DEPOIS

```
# Pega a posição do usuário-alvo dentro da matriz
user_pos = np.where(user_index == USER_ID)[0][0]  # linha do usuário-alvo na matriz
...
# Seleciona os top N_RECOMENDACOES filmes recomendados
top_ids = sorted(pontuacao_final, key=pontuacao_final.get, reverse=True)[:N_RECOMENDACOES]  # IDs dos
    filmes com maior nota estimada
```